Doc.AI

Agentic Medical RAG Assistant — AMD AI DevMaster Hackathon, Track 2: Development and Local Deployment of Private AI Agents

1. Application Scenario

Doc.AI is a private, locally-deployable AI medical assistant that helps users reason through symptoms conversationally, with two complementary knowledge sources: a curated local medical knowledge base and live, trusted web search for guidance that changes over time (treatment protocols, current outbreak information, updated clinical guidance). The system asks clarifying questions across multiple turns, detects emergency symptoms and escalates accordingly, and can generate a downloadable summary report of the conversation.

Target users are individuals seeking a first-pass understanding of their symptoms before deciding whether and which type of care to seek — not a replacement for professional diagnosis, which is stated explicitly to the user throughout the interface.

2. System Architecture

The system is a single FastAPI backend serving a lightweight HTML/CSS/JS frontend. The backend exposes three endpoints: **/chat** (main agent loop), **/report** (session summarization), and **/** (health check reporting active provider/model).

Request flow for /chat:

1. User message received, along with full session history.
2. Local retrieval: FAISS similarity search over a 50-document curated medical knowledge base (condition-specific dietary/lifestyle guidance), embedded with sentence-transformers/all-MiniLM-L6-v2.
3. Routing decision: a lightweight LLM call evaluates whether the retrieved local context is sufficient, or whether the query needs current/specific information from the web. Returns a structured JSON decision plus a search query if needed.
4. Conditional web search: if triggered, a Tavily API call retrieves content restricted to a trusted-domain allowlist (who.int, nih.gov, cdc.gov, mayoclinic.org, medlineplus.gov, ncbi.nlm.nih.gov, pubmed.ncbi.nlm.nih.gov) — general open-web search is intentionally excluded to keep medical claims traceable to reputable sources.
5. Answer synthesis: local + web context (when present) is combined into the system prompt, and the main LLM call generates the final conversational response, continuing the multi-turn symptom-clarification protocol.
6. The response returns to the frontend along with a used_web_search flag and timing metadata.

LLM provider abstraction:

The backend uses a single OpenAI-compatible client, selected at startup via a PROVIDER environment variable. This allows identical application logic to run unmodified against either Groq's cloud API (used during development for fast iteration) or a self-hosted vLLM endpoint running on AMD Radeon Cloud (used for the deployed, scored submission) — switching providers requires changing only environment variables, not code.

```
PROVIDER=radeon
RADEON_BASE_URL=https://<instance>/spaces/<id>/8000/v1
RADEON_API_KEY=<key>
```

3. Core Capabilities

Track 2 requires a minimum of 2 of the following 5 capabilities. Doc.AI implements 4:

- **Local knowledge retrieval (RAG)** — FAISS vector store over 50 curated medical condition documents.
- **Tool invocation** — live web search via the Tavily API, restricted to a trusted medical-domain allowlist.
- **Multi-step task planning** — a distinct routing/decision step precedes the final answer, determining whether external retrieval is needed before synthesis, rather than a single fixed pipeline.
- **Local multi-turn memory** — full conversation history is retained per session and passed to every subsequent call, enabling coherent multi-turn symptom clarification.

(Permission control / privacy mechanism was not implemented in this submission window.)

4. Model & Local Deployment Plan

The deployed model is **Qwen2.5-7B-Instruct-AWQ**, a 4-bit AWQ-quantized instruction-tuned model, chosen to balance response quality against load time and memory footprint on a single shared Radeon GPU instance within a constrained compute-credit budget.

Deployment was performed via vLLM on AMD Radeon Cloud, using the ROCm vLLM-dev (Navi) container image (ROCm 7.2.1, PyTorch 2.9, vLLM 0.16.0), served as an OpenAI-compatible endpoint:

```
vllm serve Qwen/Qwen2.5-7B-Instruct-AWQ --host 0.0.0.0 --port 8000
```

This produces a dedicated OpenAI-compatible base URL and API key, requiring no changes to application code — only the environment-variable switch described above.

5. Inference Speed Optimization & Benchmark

To isolate variables cleanly, benchmark questions were each asked in a fresh session (empty conversation history), so only the presence/absence of the web-search step and the underlying provider differ between runs. Timing is measured server-side, start to finish of the full /chat request (including routing decision, optional web search, and final answer generation).

Question	Provider	Web search used	Response time
Diabetes dietary guidance (local RAG only)	Groq (cloud API)	No	1.35s
Diabetes dietary guidance (local RAG only)	Radeon (local, quantized)	No	14.36s
Latest asthma treatment guidelines	Groq (cloud API)	Yes	3.78s
Latest asthma treatment guidelines	Radeon (local, quantized)	Yes	19.17s

Isolating the web-search step specifically (Radeon, same session-fresh conditions): adding live trusted-source retrieval added approximately **4.8 seconds** (14.36s → 19.17s) on top of the baseline local-RAG response.

Groq's lower absolute latency reflects its purpose-built LPU inference hardware, optimized specifically for fast token generation, and is not a like-for-like comparison against a general-purpose GPU instance running a quantized model. The meaningful result of this benchmark is not "faster than Groq" but that the identical agentic architecture — local RAG, agentic routing, trusted web search, multi-turn memory — runs correctly and fully locally on AMD Radeon GPU via ROCm and vLLM, with quantization (AWQ, 4-bit) applied specifically to keep the model's memory footprint and load time practical on a single shared instance.

6. Notable Implementation Details & Findings

- A trusted-domain allowlist was applied to web search specifically because this is a medical use case: unrestricted open-web search results are not an acceptable retrieval source for health guidance, regardless of the router's confidence.
- The routing model occasionally returned need_web_search=true with an empty search query under real deployment conditions; a fallback was added so the router substitutes the user's original message as the search query whenever this occurs, preventing a silent failure of the web-search tool.
- All external LLM/tool calls (router decision, web search) are wrapped in exception handling so that a transient network or parsing failure degrades gracefully to a local-only answer rather than returning a server error to the user.

7. Final Deliverables

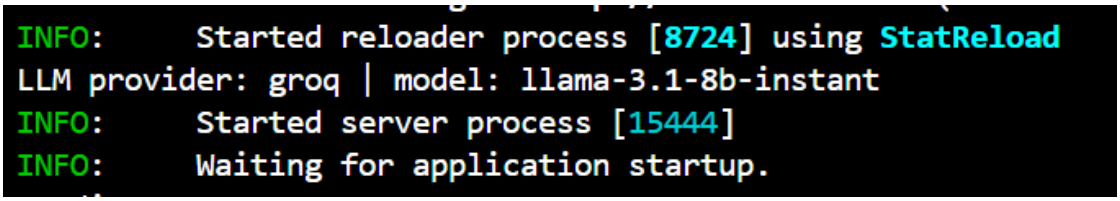
- FastAPI backend (backend.py) implementing the agentic RAG pipeline described above.
- Static HTML/CSS/JS frontend (index.html, style.css, script.js) providing a multi-session chat interface and PDF report export.
- Local knowledge base (medical_data/) of 50 curated condition documents.
- README.md with environment setup, dependency list, and startup instructions.

- Demonstration video showing the complete workflow running against the AMD Radeon-deployed model.

8. Appendix: Terminal Evidence for Benchmark Claims

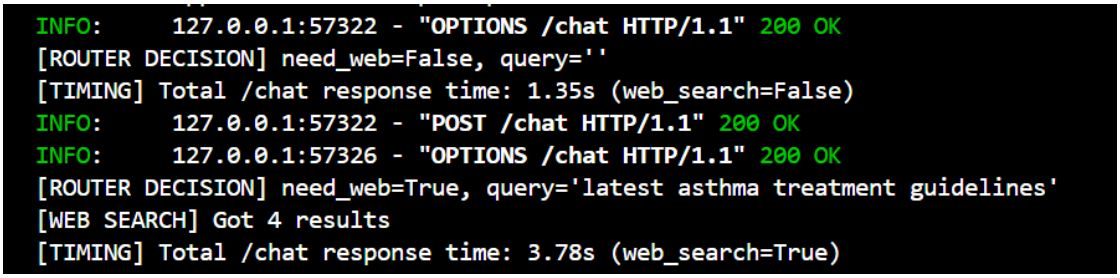
The following are unedited terminal screenshots captured during the benchmark runs referenced in Section 5, included to support the reported figures with primary evidence rather than summarized claims alone.

A. Groq provider — startup confirmation



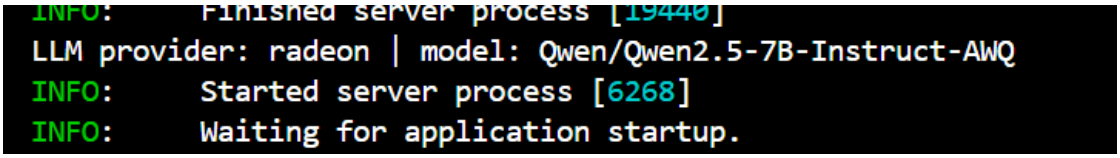
Backend startup log confirming PROVIDER=groq and model llama-3.1-8b-instant active before the Groq-side benchmark questions were run.

B. Groq provider — benchmark timing



Router decision and response-time logs for the two fresh-session benchmark questions (local RAG only: 1.35s; web-search triggered: 3.78s) on Groq.

C. Radeon provider — startup confirmation



Backend startup log confirming PROVIDER=radeon and model Qwen/Qwen2.5-7B-Instruct-AWQ active, connected to the self-hosted vLLM endpoint on AMD Radeon Cloud.

D. Radeon provider — benchmark timing



Router decision and response-time logs for the same two fresh-session benchmark questions (local RAG only: 14.36s; web-search triggered: 19.17s) on the Radeon-deployed model, confirming both the successful deployment and the isolated web-search cost of ~4.8 seconds referenced in Section 5.